

Detecting Co Travelers for Coordinated Trips with Time Expanded Interaction Graphs

CSCI 8715: Spatial Data Science Research

Abhay Shashidhara , Chaitanya Kadam(G4)

CP10-FINAL REPORT

1. Introduction

Location based social networks record when and where users visit different places. These check in records can reveal useful movement behavior when they are studied across many users and many locations. One important pattern is repeated proximity, where the same users appear near each other across different times or places.

This problem is useful because repeated proximity can help study group movement, community structure, social behavior, and public health related co presence. However, the task is not simple. Real check in data is incomplete because users do not record every visit. It is also noisy because popular venues such as malls, campuses, airports, and restaurants can create accidental co occurrences between unrelated users.

Another challenge is the lack of direct co travel labels. Datasets such as Gowalla and Brightkite provide check ins and social links, but they do not directly say which users actually traveled together. Because of this, the project needs a representation that can separate stronger repeated proximity from weak one time overlaps. To address this, the project uses a graph based approach. Users are treated as nodes, and repeated spatiotemporal encounters are treated as weighted edges. This allows the analysis to move beyond simple shared visits and focus on stronger proximity structure between users.

2. Related Work and Limitations

Cheng et al. [1] studied friendship inference from check in data in location based social networks. Their work shows that mobility patterns can provide useful signals for predicting whether two users may be socially connected. This is important because it shows that check in data can be used for relationship analysis, not only for recording user visits.

Pham et al. [2] studied social strength inference from spatiotemporal data. Their work shows that repeated meetings should not all be treated equally. A meeting at a common crowded place may be less informative than repeated meetings at more specific locations. This supports the idea that the context and quality of an encounter matter when estimating user relationships.

These studies show that raw co occurrence counts have limitations. Simply counting how many times two users appear near each other can create false signals, especially in public venues. Existing work mainly focuses on pairwise friendship or social strength inference. In contrast, this project focuses on building a graph representation that can also support group level proximity analysis.

2.1 Our Contributions

This project builds on earlier work on friendship inference and social strength estimation, but shifts the focus from only pairwise prediction to graph based repeated proximity modeling.

As shown in Figure 1, the proposed representation contains users as nodes, repeated spatiotemporal encounters as weighted edges, and closely connected regions as possible user groups. The edge labels represent different levels of proximity strength, while the dashed regions show how strongly connected users can form graph partitions. This figure illustrates the main idea of converting raw check in records into a structured proximity graph.

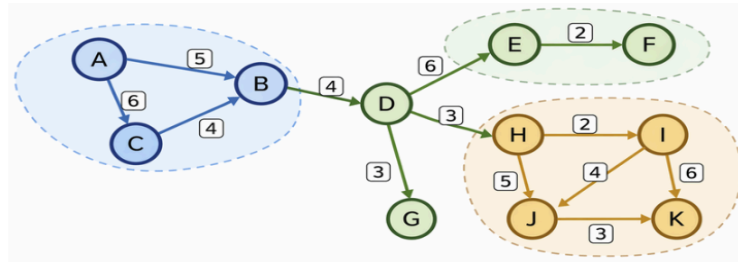


Figure 1

The main contribution of this project is the construction of a Temporal Detailed People Proximity Graph from noisy check in data. The graph edge weights combine encounter count, encounter strength, distinct days, and shared venues. This helps give higher importance to repeated and meaningful proximity rather than treating every shared visit equally. After the graph is built, it is filtered and partitioned using PyMetis to study larger proximity based communities

3. Problem Statement

Input: The input for this project is user check in records collected from location based social networking services. Each check in record contains a user ID, timestamp, latitude, longitude, and venue ID. These values help identify who visited a place, when they visited it, and where the visit happened.

Proximity Definition: Proximity is defined using spatial and temporal constraints. A user pair is considered close only if their check ins fall within a selected time window and spatial distance threshold. This helps avoid treating all shared venue visits as meaningful encounters.

Output: The output is a spatio temporal graph of people. In this graph, users are represented as nodes, and repeated proximity between users is represented as weighted edges. The graph is used to study possible co traveller patterns and group level structure.

Objective: The main objective is to reduce approximation errors while identifying co traveller patterns and user groups. Instead of relying only on simple co occurrence, the project tries to capture repeated and stronger proximity patterns across time and space.

Constraint: Real world check in data such as Gowalla and Brightkite is sparse and noisy. Users do not check in at every place they visit, and popular venues can create accidental co occurrences. These datasets also do not provide direct ground truth co travel labels, which makes evaluation and interpretation challenging

4. Proposed Method: Temporal Detailed People Proximity Graph

The proposed method converts raw location based check in records into a weighted user proximity graph. The main idea is to identify repeated spatiotemporal encounters between users and use those encounters to build a graph that can represent stronger proximity patterns and possible user groups. The complete pipeline starts with preprocessing the check in data, then creates venue time events, aggregates user pair features, assigns edge weights, filters weak graph edges, and finally partitions the graph using PyMetis.

4.1 Data Preprocessing

The first step is to clean and prepare the raw check in records. Each record contains a user ID, timestamp, latitude, longitude, and venue ID. Since the original datasets contain many users and check ins, the project first selects the most active users for analysis. This keeps the experiment manageable while still preserving users with enough mobility activity to form repeated proximity patterns.

After selecting the users, the check in records are filtered and standardized. Timestamps are converted into a consistent format, and location fields are kept for spatial comparison. Records that do not belong to the selected user set are removed. This step creates a cleaner input table that can be used for event construction.

4.2 Venue Time Event Construction

After preprocessing, the check ins are grouped into venue time events. A venue time event contains users who checked in at the same venue within a selected time window. In this project, the time bin is set to one hour. This means that check ins at the same venue during the same time interval are treated as possible proximity candidates.

However, being in the same venue time bucket is not enough. The method also checks whether users satisfy the spatial distance and time gap conditions. This prevents unrelated users from being grouped together only because they visited the same popular venue. The event construction step therefore creates candidate groups where users are close in both space and time.

For example, if three users check in at the same coffee shop between 10:00 and 11:00 AM, they form a possible venue time event. If their locations are within the distance threshold, valid user pairs are created from that event. If a user is too far away or outside the time window, that user is not included in the valid proximity pair.

4.3 User Pair Feature Aggregation

Once valid proximity events are created, the method converts event level encounters into user pair features. For every pair of users, the system counts how often they appear together and summarizes the quality of their repeated encounters.

The aggregated features include encounter count, encounter strength, number of distinct days, and number of shared venues. Encounter count captures how many times two users appeared close to each other. Encounter strength captures how strong those encounters were based on spatial and temporal closeness. Distinct days help identify whether the pair repeatedly appears together across time instead of only once. Shared venues help capture whether the users meet across multiple places.

This step is important because it changes the data from individual check in records into relationship level information. Instead of treating every check in separately, the method builds a summary of repeated proximity for each user pair

4.4 Edge Weighting

After user pair features are computed, each valid user pair is assigned an edge weight. The edge weight represents the strength of repeated spatiotemporal proximity between two users. A higher weight means that the users have stronger repeated proximity based on frequency, encounter quality, distinct days, and shared venues.

$$W_{ij} \propto C_{ij} + S_{ij} + D_{ij} + V_{ij}$$

Here, C_{ij} represents the encounter count between users i and j . S_{ij} represents the overall encounter strength. D_{ij} represents the number of distinct days on which the two users appeared close to each other. V_{ij} represents the number of shared venues between the two users.

A higher w_{ij} value means that the user pair has stronger repeated proximity across time and space. This is different from using only a simple co occurrence count. A pair that appears together many times at crowded public places should not automatically be treated as more meaningful than a pair that appears together repeatedly in smaller and more specific contexts.

By combining encounter count, encounter strength, distinct days, and shared venues, the edge weight gives a more balanced signal of proximity strength. The final weighted graph is the Temporal Detailed People Proximity Graph. In this graph, users are represented as nodes, and weighted edges represent repeated proximity between users.

4.5 Graph Partitioning using PyMetis

Since this is a node partitioning problem, the filtered TDPPG is divided into smaller user groups using PyMetis. PyMetis tries to keep stronger edges inside the same partition and cut weaker edges between partitions. This helps identify group level proximity patterns instead of only studying individual user pairs. Figure 2 shows this process with six users. Users U1, U2, and U3 form one strong group, while U4, U5, and U6 form another strong group. The edge between U3 and U6 has weight 0.29, which is below the threshold of 0.30, so it is removed as a weak cross edge. After filtering, PyMetis creates two partitions. Partition P1 contains U1, U2, and U3, and Partition P2 contains U4, U5, and U6. This shows how users with stronger repeated proximity are kept together, while weaker connections are separated. These partitions can then be used to study possible co travel, repeated co presence, and group movement behavior.

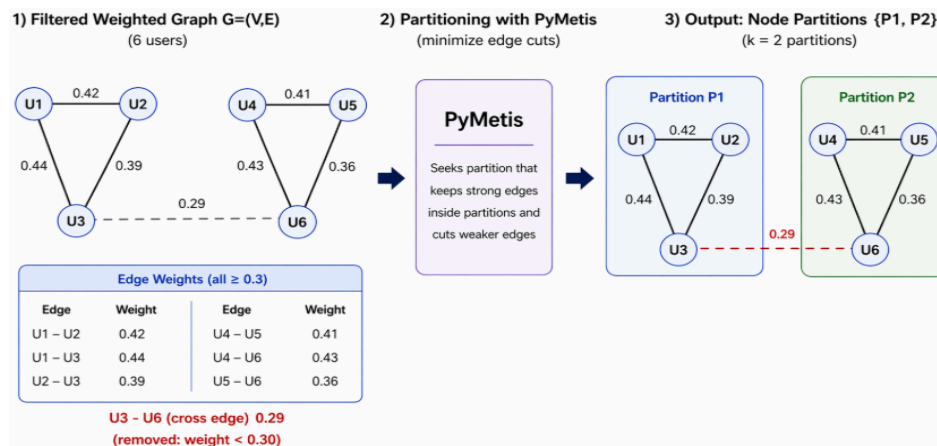


Figure 2 : Example of graph filtering and PyMetis partitioning

5 Results and Analysis

The results are analyzed in three parts. First, the graph structure is observed before partitioning to understand the raw TDPPG connectivity. Second, the graph is analyzed after PyMetis partitioning to study how users are grouped. Finally, ranking and partition quality metrics are used to compare Gowalla and Brightkite.

5.1 Graph Structure Before Partitioning

Before partitioning, both datasets form weighted TDPPG graphs from repeated check in interactions. Figure 3 shows the graph structure for Gowalla and Brightkite before PyMetis is applied, where users are nodes and edges represent repeated proximity. In Figure 3, Gowalla appears more compact and continuous, showing a stronger connected proximity structure. Brightkite appears more spread out and circular, with many visible links around the graph. This

suggests that both datasets create repeated proximity connections, but Brightkite may contain more noisy or weaker co occurrence links compared to Gowalla.

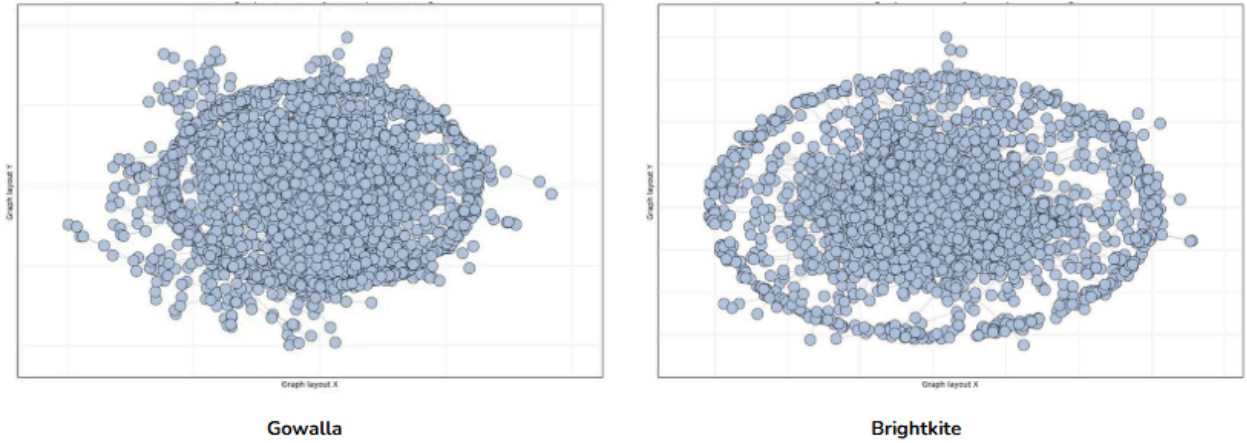


Figure 3. TDPPG graph structure before PyMetis partitioning for Gowalla and Brightkite.

5.2 Graph Structure After Partitioning

After the TDPPG is constructed, PyMetis is applied to divide the graph into smaller user groups. Figures 4 and 5 show the graph structures for Gowalla and Brightkite after partitioning. Each colored region represents a different partition, and users inside the same partition are expected to have stronger internal proximity connections. Figure 4 shows the Gowalla graph after partitioning. The graph appears more compact and clearly grouped, suggesting that Gowalla has cleaner and more meaningful repeated proximity links. Figure 5 shows the Brightkite graph after partitioning. Compared to Gowalla, Brightkite appears more spread out and fragmented, with many connections across the graph. The graph statistics after partitioning also support this difference. Gowalla has 6,826 partitioned nodes and 21,604 partitioned edges, while Brightkite has 6,010 partitioned nodes and 53,441 partitioned edges. Brightkite has fewer nodes but many more edges, which explains its higher density. However, higher edge density does not always mean better graph quality.

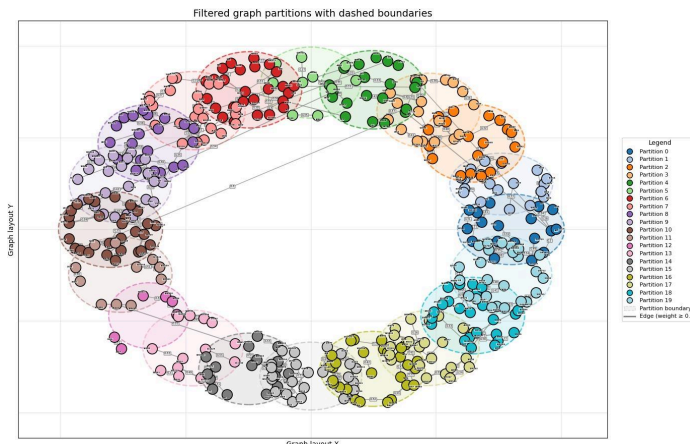


Figure 4: Gowalla

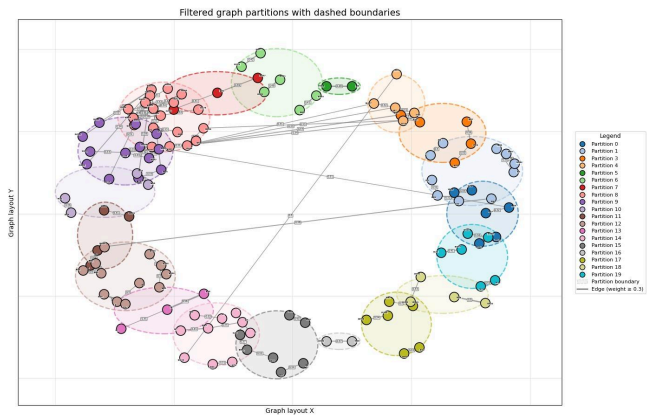


Figure 5 : Brightkite

6 Metric Evaluation

The metric evaluation compares Gowalla and Brightkite using ranking metrics and partition quality. Figure 6 shows the PR and ROC curves for both datasets, with Brightkite shown on the top row and Gowalla shown on the bottom row. The PR curves compare precision and recall, while the ROC curves compare true positive rate and false positive rate.

From Figure 6, Gowalla performs better in identifying meaningful repeated proximity links. Gowalla maintains higher precision across recall, while Brightkite's precision drops faster, suggesting more noisy proximity links. The values also support this trend. Gowalla achieves a ROC-AUC of 0.7588 and PR-AUC of 0.4506, while Brightkite achieves 0.5946 and 0.0945. Gowalla also performs much better in Precision@K, with 0.990 at P@100, 0.994 at P@500, and 0.989 at P@1000.

The same partition social edge rate is also higher for Gowalla, 0.2037 compared to 0.1405 for Brightkite. This means socially linked users are more likely to be grouped together in Gowalla after PyMetis. Overall, Figure 6 and the metric values show that Gowalla produces cleaner and more reliable proximity patterns, while Brightkite remains denser but noisier.

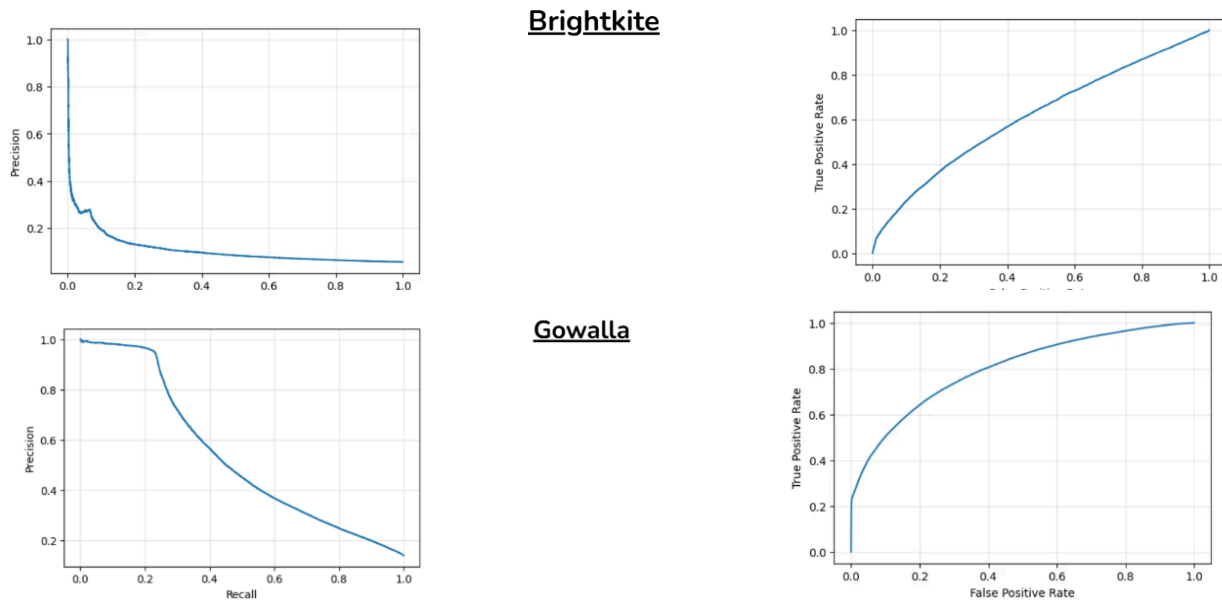


Figure 6. PR and ROC curve comparison for Brightkite and Gowalla respectively

7. Sensitivity Analysis

Sensitivity analysis was conducted to understand how the TDPPG pipeline changes when the main proximity parameters are varied. The three tested parameters were maximum distance, maximum time gap, and maximum event group size. These parameters decide how strict the system is when forming valid proximity events from check in data. The detailed results are shown separately for Gowalla in Table 1 and Brightkite in Table 2.

Parameter	Value	ROC-AUC	PR-AUC	P@100	P@500	P@1000	Graph edges
Max Distance (m)	75	0.753188	0.449094	0.99	0.994	0.989	58,927
	150	0.758812	0.450552	0.99	0.994	0.989	61,724
	300	0.758812	0.450552	0.99	0.994	0.989	61,724
Max Time Gap (s)	1800	0.758580	0.443728	0.99	0.994	0.991	50,277
	3600	0.758812	0.450552	0.99	0.994	0.989	61,724
	7200	0.758812	0.450552	0.99	0.994	0.989	61,724
Max Event Group Size	10	0.762357	0.487762	0.99	0.994	0.989	48,328
	25	0.758812	0.450552	0.99	0.994	0.989	61,724
	50	0.761133	0.424528	0.99	0.994	0.989	75,151

Table 1: Gowalla

Parameter	Value	ROC-AUC	PR-AUC	P@100	P@500	P@1000	Graph edges
Max Distance (m)	75	0.588414	0.092937	0.44	0.280	0.246	152,458
	150	0.594593	0.094495	0.44	0.284	0.248	152,642
	300	0.594593	0.094495	0.44	0.284	0.248	152,642
Max Time Gap (s)	1800	0.598848	0.104441	0.55	0.360	0.330	129,454
	3600	0.594593	0.094495	0.44	0.284	0.248	152,642
	7200	0.594593	0.094495	0.44	0.284	0.248	152,642
Max Event Group Size	10	0.611367	0.206428	0.74	0.634	0.558	66,439
	25	0.594593	0.094495	0.44	0.284	0.248	152,642
	50	0.577747	0.064176	0.25	0.136	0.110	193,723

Table 2: Brightkite

7.1 Distance Threshold Sensitivity

The distance threshold controls how far apart two users can be while still being considered close. As shown in Tables 1 and 2, the tested values were 75 m, 150 m, and 300 m. For Gowalla, the ROC-AUC stayed close to 0.7588 and the PR-AUC stayed around 0.45. For Brightkite, the ROC-AUC stayed around 0.5946 and the PR-AUC stayed around 0.0945. Precision values also changed very little. This shows that distance threshold was not the most sensitive parameter in this experiment. A value of 150 m was selected because it gives a balanced spatial range without being too strict or too loose.

7.2 Time Gap Sensitivity

The time gap threshold controls how close two check-ins must be in time to form a proximity event. As shown in Tables 1 and 2, the tested values were 1800 seconds, 3600 seconds, and 7200 seconds. For Gowalla, the results were almost unchanged across the three time gaps. For Brightkite, 1800 seconds performed slightly better, but the improvement was small. Larger time gaps added more edges but did not improve ranking quality. The final selected time gap was 3600 seconds, or one hour, because it captures realistic check-in delays without adding too many unrelated encounters.

7.3 Event Group Size Sensitivity

Maximum event group size was the most sensitive parameter. It controls the largest number of users allowed in one proximity event. As shown in Tables 1 and 2, the tested values were 10, 25, and 50. For Gowalla, group size 10 gave the best PR-AUC, while group size 25 still gave strong precision values. When the group size increased to 50, the graph became denser, but PR-AUC dropped. For Brightkite, the effect was stronger. Group size 10 performed best, while larger groups added many edges but reduced ranking quality. This shows that large venue time events can introduce noisy user pairs. Limiting event group size helps keep the graph focused on smaller and more meaningful proximity patterns.

Based on the sensitivity results in Table 1 and Table 2, the final experiment used a maximum distance of 150 m, a maximum time gap of 3600 seconds, a minimum event size of 2 users, a maximum event group size of 25 users, and 20 graph partitions. These settings were chosen to balance graph quality, noise reduction, and interpretability. The 150 m distance threshold gives a stable spatial radius, while the 3600 second time gap captures realistic temporal overlap without becoming too loose. The minimum event size of 2 allows pairwise proximity events, and the maximum group size of 25 helps reduce noisy large venue groups while still keeping enough valid events. Finally, 20 partitions provide interpretable graph communities for group level analysis.

8. Runtime and Scalability Analysis

Table 3 shows the runtime for each major step of the TDPPG pipeline on Gowalla and Brightkite. The runtime was evaluated by measuring the execution time of top user selection, filtered check in loading, pair event construction, feature aggregation, and PyMetis partitioning.

The main finding is that pair event construction is the bottleneck. It takes 177.05 seconds for Gowalla and 190.51 seconds for Brightkite because venue time events can create many candidate user pairs. In contrast, PyMetis partitioning is very fast, taking only 0.14 seconds for Gowalla and 0.19 seconds for Brightkite. This shows that scalability mainly depends on pair generation, not partitioning. Future improvements should focus on early filtering, spatial indexing, parallel processing, and pruning weak candidate pairs.

Step	Gowalla (s)	Brightkite (s)
Top-user selection	27.84	9.67
Load filtered check-ins	20.63	28.30
Build pair events	177.05	190.51
Aggregate pair features	0.23	0.69
Partition graph (PyMetis)	0.14	0.19

Table 3 : Runtime analysis for both the datasets

9. Limitations and Future Work

This project has a few important limitations. First, the datasets do not provide direct ground truth co travel labels. Friendship links were used as an indirect validation signal, but friendship is not always the same as co travel. Second, the experiments were limited to Gowalla and Brightkite, which are older location based social network datasets. Because of this, the results may not fully represent newer mobility behavior. Third, the current work does not include full baseline comparisons. Comparing TDPPG with simple co occurrence, frequency based ranking, distance only methods, or existing co traveler detection methods would make the evaluation stronger. The method also depends on manually selected thresholds, and different datasets may require different settings even though sensitivity analysis helps justify the chosen values.

Future work can address these limitations in several ways. The method should be tested on newer and more diverse datasets, and synthetic data could be used to create known ground truth co travel groups for controlled evaluation under noise, sparsity, and missing data. Another direction is to move from group detection to relationship inference, where the model tries to understand whether users are friends, commuters, coworkers, tourists, or people following similar routines. A hypergraph based TDPPG could also be explored because real co travel often happens in groups rather than only between pairs. Finally, scalability can be improved by optimizing pair event construction through faster indexing, pruning large venue time buckets, and reducing unnecessary pair comparisons.

10. Conclusion

This project proposed a Temporal Detailed People Proximity Graph for modeling repeated proximity from location-based check-in data. The method converts raw check-ins into a weighted user-user graph and partitions the graph into possible proximity-based communities. The experiments showed that Gowalla produced stronger validation results, while Brightkite produced a denser but noisier graph. Sensitivity analysis showed that maximum event group size had the largest effect on performance, while distance and time gap were more stable. Runtime analysis showed that pair-event construction is the main computational bottleneck, while PyMetis partitioning is fast. Overall, TDPPG provides a useful foundation for studying repeated proximity and possible co-travel patterns, but future work should add stronger baselines, newer datasets, synthetic ground truth, and better scalability improvements.

References

- [1] Cheng, Ran & Pang, Jun & Zhang, Yang. (2015). Inferring Friendship from Check-in Data of Location-Based Social Networks. 1284-1291. 10.1145/2808797.2808884.
- [2] Huy Pham, Cyrus Shahabi, and Yan Liu. 2016. Inferring Social Strength from Spatiotemporal Data. ACM Trans. Database Syst. 41, 1, Article 7 (April 2016), 47 pages. <https://doi.org/10.1145/2877200>
- [3] J. Leskovec and A. Krevl, "SNAP Datasets: Stanford Large Network Dataset Collection." Stanford University. ([SNAP](#))

- [4] SNAP, "Gowalla location-based online social network dataset." Stanford Network Analysis Project. ([SNAP](#))
- [5] SNAP, "Brightkite location-based online social network dataset." Stanford Network Analysis Project. ([SNAP](#))
- [6] J. Leskovec and R. Sosič, "SNAP: A General-Purpose Network Analysis and Graph-Mining Library," ACM Transactions on Intelligent Systems and Technology, 2016. ([PubMed Central](#))
- [7] G. Karypis and V. Kumar, "Multilevel k-way partitioning scheme for irregular graphs," Journal of Parallel and Distributed Computing, 1998. ([Springer Nature Link](#))
- [8] PyMetis, "PyMetis: A Python Wrapper for METIS." ([GitHub](#))